Ball-by-Ball Win Probability: viva cheat sheet

30-second pitch. Cricket matches turn on single balls, but common tools only use overs and wickets. We built a Hadoop + Spark + Hive pipeline over 22,818 Cricsheet matches (11.5M deliveries) that gives the chasing team's win probability after **every ball**, trained on older seasons and tested only on newer ones. All three synopsis targets are met. We also measure how much each ball moved the odds, check the percentages are honest, and show it all in a dashboard.

Numbers to remember (held-out seasons 2021/22 onward, 1,201,560 rows)

Item	Value	Item	Value
Matches / deliveries	22,818 / 11,492,709	Targets (synopsis)	AUC >= 0.82, Brier <= 0.18, Log-Loss <= 0.50
Chase rows / chases	2,565,140 / 17,711	Production GBT	AUC 0.913, Brier 0.120, Log-Loss 0.373, acc 82.5%
Train / test rows	1,363,580 / 1,201,560	Logistic Regression	AUC 0.914, Brier 0.119, Log-Loss 0.370 (best, by a hair)
MapReduce output	33,107 rows, 0 mismatches vs Spark	Gender/level features	Log-Loss -0.0053, interval [-0.0073, -0.0034]
League dot-ball rate	47.54%	Cluster	10 containers, 1 data node, 1 laptop

Pipeline in one line

Raw JSON → **HDFS** → **Spark flatten** (Parquet) → **MapReduce** bowler stats + **Hive** SQL → **Spark features** (state, momentum, form) → **train** (before 2021/22) → **score** every chase ball → **calibrate + WPA + compare + importance** → **dashboard**.

Why each tool

Tool	Why
Docker Compose	Full multi-service cluster on one laptop, rebuilt with one command (native Hadoop on Windows is painful)
HDFS / YARN	Reliable storage for 3.8 GB of files; YARN schedules the jobs. A course requirement
MapReduce	Bowler stats are a one-pass "add up per key" job, exactly its purpose. Cross-checked in Spark
Hive	Anyone can query results in SQL (needs PostgreSQL as its metastore)
Spark / MLlib	In-memory joins, windows and model training at scale, far better than MapReduce for multi-step work
Parquet	Compact, fast columnar format
Streamlit + Altair	Interactive dashboard in plain Python

Key terms in one line each

Chronological split: train on the past, test on the future (a random split leaks same-match balls).
Leakage: the model sees information it would not have; form windows end before the current match.
AUC: ranks winners above losers (0.5 = guess, 1 = perfect).
Brier / **Log-Loss**: penalties for wrong / confidently wrong probabilities; lower is better.

Calibration: does "70%" mean about 70%? **ECE**: average gap, by bucket.
Isotonic: step-function correction (helped); Platt did not.
WPA: win probability after a ball minus before it.
Permutation importance: scramble a feature group, see how much worse the model gets.
Confidence interval excludes 0: the gain is real, not noise.

Five answers to have ready

What did you build? A reproducible 10-container Hadoop/Spark/Hive pipeline that gives ball-by-ball chase win probability, with calibration, WPA, model comparison, importance and a five-view dashboard.

Why trust the numbers? Date-based split, leakage guards (brute-force checked), independent recomputation, intervals and placebo controls, and negative results kept.

Is GBT the best? No. Logistic Regression is as good or slightly better (wins 8 of 10 held-out seasons); we say so. Nothing was tuned.

What did you discover? The model lacked gender and level context. Adding two features gave a measured gain and cut the women's calibration gap by about 60%.

Is it really distributed? The architecture is; the deployment is one data node on one laptop, so we show no cluster speed-up or fault tolerance.

Disclose honestly

No LSTM, XGBoost, SVM or ensemble (3 models compared). Opponent and venue features tested but not in production; matchups are a lookup only. Importance is global, not per prediction. No live feed. Remaining over-optimism: women's about +5 to +7 points, men's international about +4. Dashboard Hive path untested (PyHive not installed; TSV fallback runs). No automated unit tests, no user study.